Saad Waleed

Lahore, Pakistan
+92-306-4009748 | saadadmalik04@gmail.com
GitHub: github.com/SaadADMalik
LinkedIn: linkedin.com/in/saadwaleedmalik
Portfolio: saadadmalik.github.io

Applied AI Engineer

Applied AI Engineer with 3+ years building AI-powered software across LLM applications, retrieval pipelines, backend APIs, analytics dashboards, and ML-driven decision tools. Recent proof points include Rift AI (WhatsApp chief of staff for SMBs) and Peak QA (internal agentic QA platform). Strong focus on turning business workflows and unstructured data into usable products using Python, FastAPI, Flask, Streamlit, Docker, and cloud-oriented deployment. Most effective in roles that combine AI product engineering, backend development, model integration, and deployment.

Technical Skills

Languages
Python, JavaScript, TypeScript, SQL, Node.js
AI / Agents
LangGraph, OpenClaw, MCP, Claude / OpenAI APIs, agent harnesses (Actor–Critic, tool-gated turns), OpenAI SDK, ADK, RAG, embeddings, vector search (FAISS, Pinecone, Qdrant), SentenceTransformers, prompt design, XGBoost, Scikit-learn, model evaluation
Backend Development
FastAPI, Flask, .NET, REST APIs, webhooks, async workers, job queues (lease / heartbeat reclaim), Postgres checkpoints, Meta Cloud API (WhatsApp via Kapso)
Frontend / Applications
React, Next.js, Streamlit, Dash, WebSocket / WebRTC (live browser feed), Playwright
Data & Processing
PostgreSQL, SQLite, MSSQL, PDF/text preprocessing, feature engineering, JSON workflows
Infrastructure / DevOps
Docker, Docker Compose, GitHub Actions, AWS, ECS, CI/CD, logging and monitoring

Professional Experience

AI Engineer & Acting Project Manager
Evaa Pvt Ltd (Autosmart Tech's Client) May 2026 – Present

- Led the AI engineering team while also handling project management, including task planning, technical decisions, requirements, and coordination across development work
- Built and maintained AI applications using **LangGraph, Claude, OpenAI APIs, FastAPI, Playwright, PostgreSQL, and MCP**, covering agent workflows, browser automation, and business tool integrations
- Built **Peak QA**, an internal AI-powered QA platform that crawls websites, runs automated tests, streams browser activity, and generates structured bug reports with screenshot and DOM evidence
- Designed Peak QA's **Actor–Critic workflow**, where one agent performs UI actions and a separate validation step reviews potential issues before they are reported
- Implemented background workers, PostgreSQL checkpoints, job leases, heartbeat recovery, and parallel execution so long-running QA runs could resume after failures without starting over
- Built **Rift AI**, a WhatsApp-based AI assistant for SMB owners that connects to Gmail, Calendar, Slack, Shopify, Google Sheets, and other business tools through MCP and Pipedream integrations

- Designed Rift AI around separate business workspaces, dedicated WhatsApp numbers, OAuth-based access, and isolated credentials so each customer could use their own connected tools securely
- Added confirmation steps for actions such as sending emails and creating meetings, while optimizing response times through session reuse, limited tool calls, and execution timeouts
- Worked with the team from requirements through development and deployment, reviewing work, resolving technical blockers, and coordinating releases with stakeholders

AI Engineer
Autosmart Tech July 2025 – Present

- Full-stack development of mobile and backend applications for automotive workshop automation, handling customer management, vehicle tracking, job scheduling, and operational analytics
- Architected and deployed AI-integrated systems using Flutter, .NET, and OpenAI APIs serving 15+ active workshops with 200+ monthly service transactions
- Led technical design decisions for voice-first interfaces targeting multilingual (Urdu/Hinglish) mechanics with limited app literacy, achieving 84% intent accuracy through iterative prompt engineering
- Implemented secure API patterns isolating LLM services from direct database access through intent-based JSON boundaries, enabling modular feature addition without expanding AI agent privileges
- Shipped features end-to-end from requirements gathering through production deployment, including real-time job tracking, SLA monitoring, and employee labor-time analytics
- Established code review practices and CI/CD pipelines for mobile releases, reducing deployment friction and ensuring consistent quality across iterative feature rollouts

---

AI / ML Engineer
Optivex Solutions Feb 2024 – July 2025

- Built a scalable RAG system for an internal casino knowledge base with role-based access, handling GBs of continuously growing data across departments.
- Implemented a hybrid offline LLM architecture to ensure client data privacy, sending only vector embeddings to the cloud while keeping sensitive information local.
- Deployed on AWS EC2 with optimized pipelines, leveraging metadata-based caching and precise chunking to reduce latency and minimize computational costs.
- Designed efficient data processing and retrieval workflows, maintaining minimal static tokens while ensuring high query relevance for staff-facing internal systems.
- Engineered the system for scalability and maintainability**,** enabling real-time natural language querying across multiple roles without compromising security or performance.

KEY PROJECTS

Peak QA (Internal Agentic QA Platform)

- Built Peak QA, an internal QA platform where teams submit a URL and receive a structured test run live browser feed, step-by-step timeline, and verified bug reports, replacing repetitive manual exploratory passes across large marketing/product sites.
- Orchestrated an agentic test harness with LangGraph: a stateful pipeline (site discovery → site profiling → module execution → finalize) that coordinates Playwright browser sessions, Postgres checkpoints, and worker job queues so long runs can resume without re-crawling from scratch.
- Modeled QA thinking as a two-stage agent loop: an Actor plans the next UI action from page context and history; a Critic reviews bug candidates against screenshot/DOM evidence before filing so engineering gets fewer false positives and clearer repro steps.

- Automated coverage orchestration via BFS crawl and module grouping (contact, pricing, blog, login, etc.), then parallel module workers with lease/heartbeat reclaim, mirroring how QA teams split a site into checklists but running many modules concurrently under step/time/cost guardrails.
- Gave internal users live run visibility through a WebRTC/WebSocket browser feed and activity timeline, so QA/dev can watch the agent work in real time during long LLM planning windows instead of waiting for a static report at the end.
- Shipped a team-ready ops layer (Next.js admin console + FastAPI backend): scan queueing, findings triage, publishable run summaries, org-scoped credentials, and approval gates for risky actions, turning autonomous testing into a repeatable internal workflow, not a one-off script.

Rift AI (WhatsApp Chief of Staff)

- Built Rift AI as a WhatsApp chief of staff for SMB owners: they text one number to get mail triaged, meetings booked, Slack caught up, and store analytics pulled, replacing a VA plus five open tabs with a chat they already live in.
- Ran WhatsApp on a single shared business number via Baileys (linked-device session on our host), so customers only messageno QR for themwhile inbound/outbound stays a persistent socket we operate, not a Cloud API webhook stack or a phone browser session.
- Routed each owner through our worker into an isolated OpenClaw runtime (Docker per warm tenant): WhatsApp hits Baileys → we attach that tenant's warm agent → the model uses MCP tools (Gmail, Calendar, Slack, ecommerce) → the reply goes back out on the shared number.
- Treated latency as Baileys delivery + model + MCP hops, so we coalesce rapid texts into one turn, keep per-tenant gateways warm so a follow-up skips cold start, and hard-cap each agent turn so a stuck tool chain can't hang the chat.
- Treated side effects as confirm-then-act: the agent drafts the invite or email on WhatsApp, the owner replies yes, then Calendar/Gmail fireso a chief of staff that can send does not send on a guess.
- Isolated each business as its own tenant workspace, sender allowlist, and persona/contacts files, with OAuth in Pipedream Connect (not in OpenClaw/Claude), so "read my Slack / my store" never means the model holds the owner's tokens.

TrainerSource – Multi-role SaaS Platform

- Built and shipped TrainerSource, a partner portal for trainer acquisition, onboarding, access-code attribution, and commission tracking, deployed on Vercel with Supabase (Auth, Postgres, RLS) as the shared backend.
- Integrated a gated BigCommerce storefront with the portal so trainer-issued access codes unlock the store, attribute customers, and create first-sale / reorder commissions from order webhooks and reconciliation.
- Connected the Telegram bot, portal, and BigCommerce store on a shared Supabase data layer — partners manage codes and earnings in-chat or in the dashboard; customers link accounts, browse catalog, and reorder via Telegram Mini Apps.
- Built a compliance-aware Telegram concierge on Google Gemini (gemini-2.5-flash) with optional RAG over an ingested knowledge base (product/research content), so answers are grounded in retrieved chunks rather than free-form model output alone.

- Designed end-to-end trainer and customer journeys spanning application → onboarding → active selling → payouts, with admin tools for approvals, order ops, commission review, and batch payouts.
- Implemented RAG retrieval with Gemini embeddings so the bot can ground product and research Q&A in curated knowledge (papers, forums, COAs) while enforcing multi-layer compliance filters that block dosage and medical advice.

Casino Knowledge Base RAG System

- Built a scalable RAG system for an internal knowledge base across HR, finance, and operations, with role-based access control for staff containing GBs of data.
- Implemented a hybrid search pipeline combining BM25 + vector embeddings for efficient and relevant document retrieval.
- Applied query rewriting and expansion to improve retrieval coverage and relevance, followed by cross-encoder re-ranking to ensure high-precision results.
- Engineered a hybrid offline LLM architecture to maintain privacy: only vector embeddings are sent to the cloud while raw data remains local.
- Deployed on AWS EC2, optimized with metadata-based caching, precise chunking, and minimal static tokens to reduce latency and compute costs.
- Designed continuous data ingestion and retrieval workflows for incremental updates while preserving accuracy and scalability.
- Ensured secure, modular access, separating LLM query handling from database privileges, enabling safe expansion of intents and queries.

EDUCATION

BS Software Engineering (in progress)
Virtual University of Pakistan

ADP in Artificial Intelligence
Superior University

CERTIFICATIONS

- Large Language Models on AWS
- MLOps Essentials: Model Deployment and Monitoring
- MLOps Essentials: Model Development and Integration
- AWS Machine Learning / Deployment Coursework
- Micro1 Certified